

# FENIXCMS\_9

## FASE 9 - CIERRE CRITICO DE REPRODUCIBILIDAD: package-lock.json, npm ci, CI Y GATE REAL DE PRODUCCION

Repositorio: fenixcmssl-spec/moreno

Rama: main

Prioridad: CRITICA

Ejecutar en: AI Studio / GitHub. No desplegar al VPS hasta cerrar esta fase.

---

## 0. HALLAZGO REAL DE LA AUDITORIA

El repositorio ya tiene un workflow llamado Production Gate CI en .github/workflows/production-gate.yml.

Por tanto, la proxima mejora NO es crear otro Gate desde cero: hay que hacer que el Gate actual sea realmente fiable y verde.

En el commit actual se ejecutó el workflow:

Production Gate CI - run 37135805688

Commit: fead73c6520c124d664d7577981f32790c331710

El workflow fallo en el paso npm ci antes de ejecutar Prisma, lint, tests, auditorias y build.

El error real de GitHub Actions fue:

npm ci can only install packages when your package.json and package-lock.json are in sync.

Entre los problemas reportados por npm aparecen decenas de dependencias presentes en package.json pero ausentes del lockfile, ademas de versiones incompatibles dentro del lockfile, por ejemplo:

- @jridgewell/gen-mapping
- @protobufjs/pool
- balanced-match
- confbox
- dotenv
- giget
- keyv
- untyped
- dependencias modernas de ESLint
- varias dependencias nativas/optional
- conflicto de ajv
- conflicto de json-schema-traverse

Resultado:

PRODUCTION GATE = FAILED

npm ci = FAILED

tests = NO EJECUTADOS

lint = NO EJECUTADO

build = NO EJECUTADO

production audit = NO EJECUTADO

Este es el bloqueo tecnico inmediato.

---

## 1. OBJETIVO DE FENIXCMS\_9

Dejar el repositorio en un estado realmente reproducible:

package.json

+

package-lock.json

+

.nvmrc

|

v

npm ci

|

v

Prisma

```
|  
v  
audits  
|  
v  
tests  
|  
v  
lint  
|  
v  
build  
|  
v  
RELEASE GREEN
```

Regla absoluta:

SI npm ci FALLA -> NO PRODUCCION

---

## 2. FASE 9A - RECONCILIACION package.json / package-lock.json

El archivo package.json actual declara un conjunto amplio de dependencias directas, mientras que package-lock.json no refleja ese mismo conjunto de forma consistente.

NO editar package-lock.json manualmente.

El procedimiento correcto es:

1. usar exactamente la version Node definida por .nvmrc;
2. usar una version npm 10 compatible y conocida;
3. limpiar node\_modules local si existe;
4. ejecutar npm install para regenerar el lockfile de acuerdo con package.json;
5. revisar el diff;
6. ejecutar npm ci desde cero;
7. ejecutar la suite;
8. solo despues hacer commit del package-lock actualizado.

---

## 3. FASE 9B - FUENTE DE VERDAD DE DEPENDENCIAS

Definir contractualmente:

package.json = dependencias declaradas  
package-lock.json = resolucion reproducible  
.nvmrc = Node de desarrollo/CI/VPS

Prohibido:

- añadir una dependencia y no actualizar package-lock;
- borrar una dependencia de package.json dejando residuos como fuente de verdad;
- ejecutar npm install en el VPS como metodo de despliegue;
- modificar package-lock a mano.

---

## 4. FASE 9C - ELIMINAR RESIDUOS DEL LOCKFILE

Durante la regeneracion del lockfile revisar especialmente paquetes que aparecen en package-lock pero ya no estan declarados en package.json.

El caso de Firebase debe revisarse de forma especial porque FENIXCMS\_7/8 establecieron PostgreSQL como runtime y el package.json actual no debe volver a depender de Firebase para runtime.

El resultado final debe ser coherente:

```
npm ci  
npm ls --depth=0
```

debe mostrar exactamente las dependencias directas esperadas.

---

## 5. FASE 9D - VALIDACION LOCAL CON EL MISMO CONTRATO QUE CI

Ejecutar con la version indicada por .nvmrc:

```
node --version
```

```
npm --version
```

Despues:

```
rm -rf node_modules
```

```
npm ci
```

Si npm ci falla localmente, no avanzar.

Despues:

```
npx prisma validate
```

```
npx prisma generate
```

```
npm run lint
```

```
npm test
```

```
npm run audit:runtime
```

```
npm run audit:secrets
```

```
npm run audit:production
```

```
npm run build
```

---

## 6. FASE 9E - SINCRONIZACION AUTOMATICA DEL LOCKFILE

En CI mantener npm ci como instalacion oficial.

Ademas crear una validacion de lockfile reproducible que compruebe que ejecutar npm install --package-lock-only no genera cambios inesperados.

Procedimiento conceptual del job:

```
cp package-lock.json /tmp/package-lock.before
```

```
npm install --package-lock-only --ignore-scripts
```

```
diff -u /tmp/package-lock.before package-lock.json
```

Si aparecen diferencias:

```
LOCKFILE = OUT OF SYNC
```

```
JOB = FAILED
```

Esto evita que package.json y package-lock vuelvan a separarse.

El job de CI no debe hacer commit automatico del lockfile. La correccion debe pasar por una PR/commit revisable.

---

## 7. FASE 9F - FIJAR NODE Y NPM

El repositorio ya tiene .nvmrc con Node 22.

Actualizar production-gate.yml para utilizar el contrato del repositorio:

```
actions/checkout@v7
```

```
actions/setup-node@v7
```

```
node-version-file: .nvmrc
```

```
cache: npm
```

En este momento el workflow actual usa checkout@v4 y setup-node@v4 con node-version 22 fija en el YAML.

Eso no es el bloqueo principal, pero debe corregirse para que el workflow tenga una sola fuente de verdad.

Referencias oficiales verificadas:

<https://github.com/actions/setup-node/releases>

<https://github.com/actions/checkout>

---

## 8. FASE 9G - CI REAL DESPUES DE npm ci

El Production Gate actual ya tiene esta secuencia:

```
checkout
setup node
npm ci
prisma generate
lint
tests
audit:production
build
```

Una vez solucionado el lockfile, conservar esa secuencia y ampliarla con:

```
npx prisma validate
npm run audit:source-of-truth
```

si el script existe en main.

---

## 9. FASE 9H - POSTGRESQL DE CI

Una vez npm ci sea verde, añadir PostgreSQL temporal de CI.

Objetivo:

- probar Prisma con una DB limpia;
- ejecutar migraciones;
- ejecutar tests de persistencia;
- impedir que una suite pase solamente por memoria.

El CI no debe utilizar DATABASE\_URL del VPS.

---

## 10. FASE 9I - PRISMA MIGRATIONS

Ejecutar:

```
npx prisma validate
npx prisma generate
npx prisma migrate deploy
```

En la DB temporal.

Prohibido en CI y production:

```
npx prisma migrate reset
```

---

## 11. FASE 9J - PRODUCTION SOURCE OF TRUTH

El repositorio contiene:

```
scripts/audit-production-source-of-truth.ts
```

Crear un script npm dedicado:

```
audit:source-of-truth
```

y ejecutar:

```
npm run audit:source-of-truth
```

La auditoria debe quedar dentro del Gate.

---

## 12. FASE 9K - TESTS

scripts/run-all-tests.ts debe representar la suite obligatoria de produccion.

Comprobar que incluye:

- FENIXCMS\_1;
- FENIXCMS\_2;
- aislamiento multi-tenant;
- contratos API;
- precision monetaria;
- PayPal/webhooks;
- persistence;
- activation flow;
- source-of-truth;
- production readiness.

Crear tests especificos del lockfile no hace falta dentro de runtime: el check debe vivir en CI.

---

## 13. FASE 9L - DEPENDENCY AUDIT

Despues de npm ci:

npm audit --omit=dev

Definir politica:

- vulnerabilidad critica runtime = FAIL;
- vulnerabilidad alta = revisar y bloquear si afecta al runtime;
- vulnerabilidades de desarrollo = documentar o corregir segun riesgo.

No ignorar silenciosamente un audit.

---

## 14. FASE 9M - BUILD

Ejecutar:

npm run build

Solo despues de pasar:

npm ci  
prisma  
audits  
tests  
lint

El build no debe ejecutarse con dependencias instaladas de forma distinta a npm ci.

---

## 15. FASE 9N - RELEASE MANIFEST

Una vez todo pase, generar:

release-manifest.json

Campos:

application  
commit  
node  
npm  
packageLockHash  
prisma  
audits  
tests  
lint  
build  
createdAt

packageLockHash es importante para demostrar que el artefacto fue construido con exactamente la resolucion de dependencias validada.

## 16. FASE 9O - CHECK DE PACKAGE-LOCK HASH

El release debe guardar un hash SHA-256 de package-lock.json.

Ejemplo conceptual:

```
sha256sum package-lock.json
```

El hash del artefacto debe coincidir con el hash que se utilizo durante CI.

Si cambia:

RELEASE = REJECTED

## 17. FASE 9P - DEPLOY SOLO DEL ARTEFACTO VALIDADO

El VPS no debe ejecutar npm install para reconstruir dependencias.

El artefacto de CI debe ser el que se despliega.

Esto evita:

Git SHA validado

!=

node\_modules generados en VPS

## 18. FASE 9Q - WORKFLOW RELEASE

Crear o completar:

```
.github/workflows/release.yml
```

Debe:

1. recibir o identificar un SHA que tenga CI verde;
2. descargar el artefacto correspondiente;
3. comprobar manifest;
4. comprobar packageLockHash;
5. ejecutar migration segura;
6. desplegar release inmutable;
7. reiniciar systemd;
8. comprobar health;
9. ejecutar smoke tests;
10. registrar resultado.

## 19. FASE 9R - CONCURRENCY

Impedir dos releases simultaneas sobre production.

Usar una clave de concurrency equivalente a:

```
fenixcms-production
```

No cancelar automaticamente una release que ya esta cambiando la produccion salvo una estrategia controlada.

## 20. FASE 9S - HEALTH CHECK

Despues del release:

```
/api/health?type=liveness
```

```
/api/health?type=readiness
```

Readiness debe confirmar PostgreSQL.

Si readiness falla:

RELEASE = FAILED

---

## 21. FASE 9T - ROLLBACK

Si el smoke test falla:

1. no considerar el deploy exitoso;
2. volver al artefacto anterior;
3. restart;
4. health;
5. registrar rollback.

No hacer rollback destructivo de PostgreSQL automaticamente.

---

## 22. FASE 9U - NO DESPLEGAR DESDE UN WORKSPACE DISTINTO

El release debe tener un identificador unico:

commit SHA + packageLockHash

Esto impide que se compile una cosa y se despliegue otra.

---

## 23. FASE 9V - PROTECCION DE MAIN

Configurar branch protection para requerir el Production Gate CI.

No permitir merge si:

- npm ci falla;
- tests fallan;
- audit falla;
- lint falla;
- build falla.

---

## 24. FASE 9W - PROMPT MAESTRO PARA AI STUDIO

Copiar este bloque completo en AI Studio:

### INICIO FASE FENIXCMS\_9

DONDE SE EJECUTA:

AI STUDIO / repositorio fenixcmssl-spec/moreno

OBJETIVO:

Corregir el bloqueo actual de Production Gate CI.

La ejecucion real del workflow 37135805688 demostro que npm ci falla porque package.json y package-lock.json estan desincronizados.

REGLA:

NO npm ci GREEN = NO PRODUCCION

PASO 1 - AUDITORIA

Inspecciona package.json, package-lock.json, .nvmrc y production-gate.yml.

No edites el lockfile manualmente.

PASO 2 - RECONCILIACION

Usa la version Node indicada por .nvmrc.

Usa npm compatible con Node 22.

Ejecuta npm install para regenerar package-lock.json desde package.json.

Revisa el diff completo.

Elimina residuos de dependencias que ya no existan en package.json.

No introduces nuevas dependencias funcionales salvo que el codigo realmente las necesite.

PASO 3 - VALIDACION

Borra node\_modules.  
Ejecuta npm ci desde cero.  
Despues ejecuta:  
npx prisma validate  
npx prisma generate  
npm run lint  
npm test  
npm run audit:runtime  
npm run audit:secrets  
npm run audit:production  
npm run build

#### PASO 4 - LOCKFILE REPRODUCIBLE

Ejecuta npm install --package-lock-only --ignore-scripts sobre el estado ya corregido.  
Comprueba que package-lock.json no cambia.  
Si cambia, corrige la causa antes de cerrar la fase.

#### PASO 5 - CI

Actualiza production-gate.yml para usar checkout@v7 y setup-node@v7 con node-version-file=.nvmrc.  
Mantén npm ci como instalacion oficial.

#### PASO 6 - SOURCE OF TRUTH

Añade audit:source-of-truth a package.json y ejecutalo en CI.

#### PASO 7 - RELEASE

No implementes deploy VPS todavia si el Gate no esta verde.  
Primero deja CI completamente verde.

#### PASO 8 - INFORME

Entrega exactamente:

- cambio de package-lock
- diff de dependencias
- version Node/npm
- resultado npm ci
- resultado Prisma
- resultado audits
- resultado tests
- resultado lint
- resultado build
- nuevo run de Production Gate CI

No declares PASS si Production Gate CI sigue fallando.

### FIN FASE FENIXCMS\_9

---

## 25. CRITERIOS DE ACEPTACION

- [ ] package.json y package-lock.json sincronizados.
- [ ] npm ci PASS desde workspace limpio.
- [ ] no quedan paquetes declarados solo en package-lock sin justificacion.
- [ ] no faltan dependencias de package.json en package-lock.
- [ ] no hay conflictos de versiones del lockfile.
- [ ] Node 22 del .nvmrc usado por CI.
- [ ] Production Gate usa setup-node y checkout actuales soportados por el proyecto.
- [ ] Prisma validate PASS.
- [ ] Prisma generate PASS.
- [ ] npm run lint PASS.
- [ ] npm test PASS.
- [ ] npm run audit:runtime PASS.
- [ ] npm run audit:secrets PASS.
- [ ] npm run audit:production PASS.
- [ ] npm run build PASS.
- [ ] Production Gate CI PASS para el nuevo commit.
- [ ] release-manifest con packageLockHash generado.
- [ ] branch protection exige el Gate.

---

## 26. DECISION FINAL

Este es el siguiente paso antes de cualquier decision de deploy porque el propio GitHub Actions ya ha demostrado que el repositorio actual no puede reproducir sus dependencias con npm ci.

Mientras Production Gate CI falle en npm ci, FenixCMS no debe considerarse listo para staging ni produccion.

El siguiente objetivo despues de cerrar FENIXCMS\_9 sera ejecutar una validacion E2E de staging, especialmente PayPal Sandbox, health, login, activacion, tenant isolation y recovery.

FIN DEL DOCUMENTO FENIXCMS\_9